DATA STRUCTURES & ALGORITHMS · C++

Heaps & Priority Queues Solved Questions

Every problem from Part 13 of the Heaps & Priority Queues guide, in order, with a worked C++ solution, the approach behind it, and the mistake that most often breaks it.

38	3	12	115
PROBLEMS	TIERS	PATTERNS	TESTS PASSING

Prepared for **Hari** · on-campus placement preparation
Companion to **Heaps-and-Priority-Queues-Guide.pdf** · code verified against **reference-part13-solutions.cpp**
Every solution in this document was compiled with **g++ -std=c++17 -O2 -Wall** and passes its test cases.
Generated 14 October 2026

Contents

Tier 1 — Learn the mechanics

01 LC 1046 · Last Stone Weight
02 LC 703 · Kth Largest Element in a Stream
03 LC 215 · Kth Largest Element in an Array
04 LC 347 · Top K Frequent Elements
05 LC 973 · K Closest Points to Origin
06 LC 1337 · The K Weakest Rows in a Matrix
07 LC 2558 · Take Gifts From the Richest Pile
08 LC 506 · Relative Ranks
09 LC 451 · Sort Characters By Frequency
10 LC 692 · Top K Frequent Words
11 LC 23 · Merge k Sorted Lists
12 LC 378 · Kth Smallest Element in a Sorted Matrix
13 LC 1962 · Remove Stones to Minimize the Total

Tier 2 — The placement core

14 LC 295 · Find Median from Data Stream
15 LC 621 · Task Scheduler
16 LC 767 · Reorganize String
17 LC 1642 · Furthest Building You Can Reach
18 LC 1834 · Single-Threaded CPU
19 LC 373 · Find K Pairs with Smallest Sums
20 LC 355 · Design Twitter
21 LC 264 · Ugly Number II
22 LC 658 · Find K Closest Elements
23 LC 846 · Hand of Straights
24 LC 743 · Network Delay Time
25 LC 1353 · Maximum Number of Events That Can Be Attended
26 LC 1405 · Longest Happy String
27 LC 871 · Minimum Number of Refueling Stops
28 LC 1882 · Process Tasks Using Servers

Tier 3 — Harder, but they do appear

29 LC 480 · Sliding Window Median
30 LC 502 · IPO
31 LC 630 · Course Schedule III

32 LC 857 · Minimum Cost to Hire K Workers
33 LC 1383 · Maximum Performance of a Team
34 LC 632 · Smallest Range Covering Elements from K Lists
35 LC 407 · Trapping Rain Water II
36 LC 778 · Swim in Rising Water
37 LC 1675 · Minimize Deviation in Array
38 LC 218 · The Skyline Problem

Tier 1 — Learn the mechanics

Thirteen problems: the bounded heap, top-K by frequency, custom comparators and the k-way merge. When you can write 215, 347, 973 and 23 cold — and say which way each heap points — you have the mechanics.

01 LC 1046 · Last Stone Weight

EASY Pattern 7 — Combine the two best

PROBLEM

Repeatedly smash the two heaviest stones. Equal stones both vanish; otherwise the difference goes back. Return the last stone's weight, or 0.

APPROACH

The problem literally says *take the two largest, many times, while the set changes* — the definition of a heap problem. Build a max-heap from the array (O(n)), pop two, push the difference if it is non-zero.

Complexity O(n log n) time · O(n) space

C++ SOLUTION

```cpp
class Solution {
public:
    int lastStoneWeight(vector<int>& stones) {
        priority_queue<int> pq(stones.begin(), stones.end());   // max-heap, O(n) build
        while (pq.size() > 1) {
            int y = pq.top(); pq.pop();                         // heaviest
            int x = pq.top(); pq.pop();                         // second heaviest
            if (y != x) pq.push(y - x);                         // the remainder goes back
        }
        return pq.empty() ? 0 : pq.top();
    }
};
```

WATCH OUT

The loop condition is `pq.size() > 1`, so there are always two elements to pop. Popping twice under `!pq.empty()` reads `top()` on an empty heap when one stone is left — undefined behaviour. The final heap may be empty, so check before the last `top()`.

02 LC 703 · Kth Largest Element in a Stream

EASY Pattern 2 — Bounded heap, streaming

PROBLEM

Design a class that receives numbers one at a time and, after each, returns the k-th largest so far.

APPROACH

Keep a **min-heap of the k largest** values seen. Each `add` pushes, and if the heap has grown past k, pops the smallest. The top is then the weakest member of the top-k club — exactly the k-th largest.

The constructor reuses `add`, so there is one code path.

Complexity O(log k) per add · O(k) space

C++ SOLUTION

```cpp
class KthLargest {
    int k;
    priority_queue<int, vector<int>, greater<int>> pq;   // MIN-heap of the k largest
public:
    KthLargest(int k, vector<int>& nums) : k(k) {
        for (int x : nums) add(x);
    }
    int add(int val) {
        pq.push(val);
        if ((int)pq.size() > k) pq.pop();                // evict the smallest
        return pq.top();                                 // smallest of the top k
    }
};
```

WATCH OUT

Min-heap, not max-heap. A max-heap of everything gives the largest, not the k-th largest, and would need k pops per query. The initial array may have fewer than k elements — the problem guarantees the heap has k by the time `add` returns, so do not assume it in the constructor.

03 LC 215 · Kth Largest Element in an Array

MEDIUM Pattern 2 — Bounded heap

PROBLEM

Return the k-th largest element of an unsorted array (the k-th in sorted order, not the k-th distinct).

APPROACH

Stream the array through a min-heap of size k: push, and pop whenever the size exceeds k. The top at the end is the answer.

Sorting is O(n log n). The heap is O(n log k). **Quickselect** is O(n) on average — the expected follow-up — and `nth_element` is its library form.

> **Complexity** O(n log k) time · O(k) space (quickselect: O(n) average, O(n²) worst)

C++ SOLUTION

```cpp
class Solution {
public:
    int findKthLargest(vector<int>& nums, int k) {
        priority_queue<int, vector<int>, greater<int>> pq;   // min-heap, size <= k
        for (int x : nums) {
            pq.push(x);
            if ((int)pq.size() > k) pq.pop();
        }
        return pq.top();          // the weakest member of the top-k club
    }
};
```

> **WATCH OUT**
>
> Duplicates count: in `[3,2,3,1,2,4,5,5,6]` the 4th largest is 4, because both 5s count. The heap handles that for free. If you write quickselect, randomise the pivot — a fixed last-element pivot is O(n²) on sorted input, and LeetCode includes such tests.

04 LC 347 · Top K Frequent Elements

MEDIUM Pattern 3 — Top-K by frequency

PROBLEM

Return the k most frequent values, in any order.

APPROACH

Count with a hash map. Then run the size-k rule on `(frequency, value)` pairs: a **min-heap** ordered by frequency, evicting the least frequent whenever the size exceeds k.

Putting the frequency **first** in the pair means the built-in `pair` ordering compares frequencies — no comparator needed.

Complexity O(n log k) time · O(n) space for the counts

C++ SOLUTION

```cpp
class Solution {
public:
    vector<int> topKFrequent(vector<int>& nums, int k) {
        unordered_map<int, int> freq;
        for (int x : nums) freq[x]++;
        // min-heap on frequency: the top is the LEAST frequent of the current k
        priority_queue<pair<int, int>, vector<pair<int, int>>, greater<>> pq;
        for (auto& [val, f] : freq) {
            pq.push({f, val});
            if ((int)pq.size() > k) pq.pop();
        }
        vector<int> res;
        while (!pq.empty()) { res.push_back(pq.top().second); pq.pop(); }
        return res;
    }
};
```

WATCH OUT

Pushing `(value, frequency)` instead of `(frequency, value)` silently orders by value. The O(n) follow-up is **bucket sort**: `bucket[f]` holds the values with frequency f, and you walk from f = n down. Mention it after giving the heap solution.

05 LC 973 · K Closest Points to Origin

MEDIUM Pattern 2 — Max-heap of the k closest

PROBLEM

Return the k points closest to (0, 0), in any order.

APPROACH

Closest = smallest distance, and you are keeping the k **smallest** — so a **max-heap** of size k, whose top is the farthest point kept and the first to be evicted.

Compare **squared** distances: the order is the same and there is no floating point. Store the point's index with its distance.

Complexity O(n log k) time · O(k) space

C++ SOLUTION

```cpp
class Solution {
public:
    vector<vector<int>> kClosest(vector<vector<int>>& points, int k) {
        // MAX-heap on squared distance: the top is the FARTHEST of the k kept
        priority_queue<pair<long long, int>> pq;
        for (int i = 0; i < (int)points.size(); i++) {
            long long x = points[i][0], y = points[i][1];
            pq.push({x * x + y * y, i});              // no sqrt needed
            if ((int)pq.size() > k) pq.pop();
        }
        vector<vector<int>> res;
        while (!pq.empty()) { res.push_back(points[pq.top().second]); pq.pop(); }
        return res;
    }
};
```

WATCH OUT

Direction again: a min-heap of size k here would keep the k *farthest* points. And `xx + yy` with coordinates up to 10^4 fits in `int`, but in variants with larger coordinates it does not — store distances as `long long` out of habit.

06 LC 1337 · The K Weakest Rows in a Matrix

EASY Pattern 2 — Tie-break by index

PROBLEM

Each row is soldiers (1s) followed by civilians (0s). Row i is weaker than row j if it has fewer soldiers, or the same number and i < j. Return the k weakest rows, weakest first.

APPROACH

Push `(soldiers, row)` into a **max-heap** of size k. The pair ordering compares soldiers first and row index second, which is exactly the problem's weakness order — the strongest kept row is on top and gets evicted.

Pop into the result from the back so it comes out weakest first.

Complexity O(m · n + m log k) time · O(k) space (binary search per row makes counting O(log n))

C++ SOLUTION

```cpp
class Solution {
public:
    vector<int> kWeakestRows(vector<vector<int>>& mat, int k) {
        // max-heap of (soldiers, row): pair compares soldiers, then row index,
        // which is exactly the tie-break the problem asks for
        priority_queue<pair<int, int>> pq;
        for (int r = 0; r < (int)mat.size(); r++) {
            int soldiers = accumulate(mat[r].begin(), mat[r].end(), 0);
            pq.push({soldiers, r});
            if ((int)pq.size() > k) pq.pop();          // drop the strongest
        }
        vector<int> res(k);
        for (int i = k - 1; i >= 0; i--) { res[i] = pq.top().second; pq.pop(); }
        return res;                                    // weakest first
    }
};
```

WATCH OUT

The tie-break is the trap. Because the heap is a max-heap on `(soldiers, row)`, a tie on soldiers evicts the **larger** index first — which is right. If you had used the negated or swapped pair, ties would come out reversed. Popping gives strongest-first, so fill the result backwards.

07 LC 2558 · Take Gifts From the Richest Pile

EASY Pattern 1 — Take the best, put back

PROBLEM

k times: take the largest pile and leave behind the floor of its square root. Return the total remaining.

APPROACH

Max-heap of piles. Each second: pop the largest, push `floor(sqrt(top))`. Sum the heap at the end.

Complexity O(n + k log n) time · O(n) space

C++ SOLUTION

```cpp
class Solution {
public:
    long long pickGifts(vector<int>& gifts, int k) {
        priority_queue<int> pq(gifts.begin(), gifts.end());
        while (k--) {
            int top = pq.top(); pq.pop();
            pq.push((int)sqrt((double)top));          // floor of the square root
        }
        long long sum = 0;
        while (!pq.empty()) { sum += pq.top(); pq.pop(); }
        return sum;
    }
};
```

WATCH OUT

The answer can exceed `int` (n up to 10^3 piles of 10^9 each), so the return type is `long long` — sum in `long long`. `sqrt` on a `double` is exact enough for values up to 10^9; for larger values use an integer square root.

08 LC 506 · Relative Ranks

EASY Pattern 1 — Pop in order

PROBLEM

Given distinct scores, return each athlete's rank: "Gold Medal", "Silver Medal", "Bronze Medal" for the top three, and the place number as a string otherwise.

APPROACH

Push `(score, index)` into a max-heap. Popping yields athletes from first place downward, and the stored index says where to write each one's rank.

Complexity O(n log n) time · O(n) space

C++ SOLUTION

```cpp
class Solution {
public:
    vector<string> findRelativeRanks(vector<int>& score) {
        priority_queue<pair<int, int>> pq;            // (score, athlete index)
        for (int i = 0; i < (int)score.size(); i++) pq.push({score[i], i});
        vector<string> res(score.size());
        const string medal[3] = {"Gold Medal", "Silver Medal", "Bronze Medal"};
        for (int place = 1; !pq.empty(); place++) {
            int i = pq.top().second; pq.pop();
            res[i] = place <= 3 ? medal[place - 1] : to_string(place);
        }
        return res;
    }
};
```

WATCH OUT

Push the **index** with the score — the answer is indexed by athlete, not by score, and a bare score cannot tell you whose rank it is. (Sorting indices by score is equally good; the heap version practises the pattern.)

09 LC 451 · Sort Characters By Frequency

MEDIUM Pattern 3 — Frequency heap

PROBLEM

Rearrange a string so characters appear in decreasing order of frequency. Characters with equal frequency may be in any order.

APPROACH

Count characters, push `(count, char)` into a max-heap, and pop: each pop appends `count` copies of that character. `string::append(count, ch)` does the copies in one call.

Complexity O(n + σ log σ) for σ distinct characters · O(n) space

C++ SOLUTION

```cpp
class Solution {
public:
    string frequencySort(string s) {
        unordered_map<char, int> freq;
        for (char c : s) freq[c]++;
        priority_queue<pair<int, char>> pq;          // most frequent on top
        for (auto& [c, f] : freq) pq.push({f, c});
        string res;
        while (!pq.empty()) {
            auto [f, c] = pq.top(); pq.pop();
            res.append(f, c);                        // f copies of c
        }
        return res;
    }
};
```

WATCH OUT

Equal characters must stay together — `"eert"` is valid, `"eret"` is not. Appending all copies of one character at once guarantees it. The problem is case-sensitive: `'A'` and `'a'` are different characters.

10 LC 692 · Top K Frequent Words

MEDIUM Pattern 3 — Custom comparator, ties

PROBLEM

Return the k most frequent words, sorted by frequency descending, and alphabetically for equal frequencies.

APPROACH

Same size-k heap as LC 347, but the free `pair` ordering gets the tie-break wrong, so write a functor. The heap must keep the **worst** candidate on top, so the comparator returns `true` when `a` is **better** than `b` (higher count, or same count and alphabetically smaller) — better candidates sink.

Pop gives worst-first; fill the result from the back.

Complexity O(n log k) time · O(n) space

C++ SOLUTION

```cpp
class Solution {
    // "a < b" for a MIN-heap must mean "a is BETTER than b" — the heap keeps the
    // worst on top so it can be evicted. Better = higher count, then smaller word.
    struct Cmp {
        bool operator()(const pair<int, string>& a, const pair<int, string>& b) const {
            if (a.first != b.first) return a.first > b.first;
            return a.second < b.second;
        }
    };
public:
    vector<string> topKFrequent(vector<string>& words, int k) {
        unordered_map<string, int> freq;
        for (auto& w : words) freq[w]++;
        priority_queue<pair<int, string>, vector<pair<int, string>>, Cmp> pq;
        for (auto& [w, f] : freq) {
            pq.push({f, w});
            if ((int)pq.size() > k) pq.pop();         // pops the WORST candidate
        }
        vector<string> res(k);
        for (int i = k - 1; i >= 0; i--) { res[i] = pq.top().second; pq.pop(); }
        return res;                                   // best first
    }
};
```

WATCH OUT

This problem exists to test the comparator rule: `cmp(a, b)` true means `a` goes **below** `b`. Test on a tie — `["b","a","c"]`, k = 2 must give `["a","b"]`. Getting either half of the comparator backwards passes the examples without ties and fails the rest.

11 LC 23 · Merge k Sorted Lists

HARD Pattern 4 — K-way merge

PROBLEM

Merge k sorted linked lists into one sorted list.

APPROACH

The next output node is the smallest of the k current heads. Keep exactly those heads in a min-heap of node pointers. Pop the smallest, append it, and push its `next` if there is one.

The lambda comparator `a->val > b->val` makes the min-heap; `decltype(cmp)` names its type and the constructor receives the lambda.

Complexity O(N log k) for N total nodes · O(k) heap

C++ SOLUTION

```cpp
class Solution {
public:
    ListNode* mergeKLists(vector<ListNode*>& lists) {
        auto cmp = [](ListNode* a, ListNode* b) { return a->val > b->val; };
        priority_queue<ListNode*, vector<ListNode*>, decltype(cmp)> pq(cmp);  // min-heap
        for (ListNode* h : lists) if (h) pq.push(h);  // one head per list
        ListNode dummy(0), *tail = &dummy;
        while (!pq.empty()) {
            ListNode* node = pq.top(); pq.pop();      // smallest head overall
            tail->next = node; tail = node;
            if (node->next) pq.push(node->next);      // its successor replaces it
        }
        return dummy.next;
    }
};
```

WATCH OUT

Skip empty lists when seeding the heap — `[[]]` is a valid input, and a `nullptr` in the heap is dereferenced by the comparator. The alternative without a heap is divide-and-conquer pairwise merging, also O(N log k); merging the lists one after another is O(N · k).

12 LC 378 · Kth Smallest Element in a Sorted Matrix

MEDIUM Pattern 4 — Rows as sorted lists

PROBLEM

Every row and column of an n × n matrix is sorted. Return the k-th smallest element.

APPROACH

Each row is a sorted list, so this is a k-way merge that stops after k outputs. Seed the min-heap with the first element of each row (only the first k rows can matter), then pop k − 1 times, each time pushing the next element of the popped row. The top is the answer.

Complexity O(k log n) time · O(n) space

C++ SOLUTION

```cpp
class Solution {
public:
    int kthSmallest(vector<vector<int>>& m, int k) {
        int n = m.size();
        // (value, row, col); every row is a sorted list -> k-way merge
        priority_queue<tuple<int, int, int>, vector<tuple<int, int, int>>, greater<>> pq;
        for (int r = 0; r < min(n, k); r++) pq.push({m[r][0], r, 0});
        while (--k) {                                  // pop k-1 times
            auto [v, r, c] = pq.top(); pq.pop();
            if (c + 1 < n) pq.push({m[r][c + 1], r, c + 1});
        }
        return get<0>(pq.top());
    }
};
```

WATCH OUT

Do not push all n² elements — that ignores the sortedness and costs O(n² log n). The other well-known solution is **binary search on the value range**, counting elements ≤ mid with a staircase walk in O(n): O(n log(max − min)). Mention it as the follow-up.

13 LC 1962 · Remove Stones to Minimize the Total

MEDIUM Pattern 1 — Take the best, put back

PROBLEM

k times, choose a pile and remove `floor(pile / 2)` stones from it. Minimise the total left.

APPROACH

Removing half of the biggest pile removes the most stones, and the choice never hurts a later one — so always pick the current maximum. Max-heap; keep a running total and subtract what each operation removes.

Complexity O(n + k log n) time · O(n) space

C++ SOLUTION

```cpp
class Solution {
public:
    int minStoneSum(vector<int>& piles, int k) {
        priority_queue<int> pq(piles.begin(), piles.end());
        int total = accumulate(piles.begin(), piles.end(), 0);
        while (k--) {
            int top = pq.top(); pq.pop();
            int removed = top / 2;                    // floor(top / 2) stones go
            total -= removed;
            pq.push(top - removed);
        }
        return total;
    }
};
```

WATCH OUT

`floor(pile / 2)` stones are **removed**, so the pile becomes `pile - pile / 2`, which is the **ceiling** of half. Pushing `pile / 2` back instead is off by one on every odd pile.

Tier 2 — The placement core

These are what actually get asked. Median from Data Stream, Task Scheduler, Reorganize String and Network Delay Time, plus the scheduling and regret-greedy patterns that turn up in every drive.

14 LC 295 · Find Median from Data Stream

HARD Pattern 5 — Two heaps

PROBLEM

Design a structure supporting `addNum` and `findMedian` over a growing stream of integers.

APPROACH

`lo` is a **max-heap** holding the smaller half; `hi` is a **min-heap** holding the larger half. Invariants: every `lo` ≤ every `hi`, and `lo` has the same size as `hi` or one more.

The three-line `addNum` keeps both without any case analysis: push into `lo`; move `lo`'s top to `hi` (restores the order); if `hi` became bigger, move its top back (restores the sizes).

Complexity O(log n) per add · O(1) per median · O(n) space

C++ SOLUTION

```cpp
class MedianFinder {
    priority_queue<int> lo;                                 // max-heap: smaller half
    priority_queue<int, vector<int>, greater<int>> hi;    // min-heap: larger half
    // invariant: every lo <= every hi, and lo.size() == hi.size() or one more
public:
    void addNum(int num) {
        lo.push(num);                          // 1. always enter through lo
        hi.push(lo.top()); lo.pop();           // 2. hand lo's largest to hi (fixes order)
        if (hi.size() > lo.size()) {           // 3. rebalance sizes
            lo.push(hi.top()); hi.pop();
        }
    }
    double findMedian() {
        if (lo.size() > hi.size()) return lo.top();
        return (lo.top() + (double)hi.top()) / 2.0;
    }
};
```

WATCH OUT

`(lo.top() + hi.top()) / 2` overflows `int` when both are near 2^{31} and truncates when they are integers. Convert to `double` **before** adding. The follow-ups — *all numbers in [0, 100]* (a count array) and *99% in [0, 100]* (counts plus two small heaps for outliers) — are common.

15 LC 621 · Task Scheduler

MEDIUM Pattern 9 — Cooldown queue

PROBLEM

Tasks are letters; identical tasks need at least n intervals between them. Each interval runs one task or idles. Return the minimum number of intervals.

APPROACH

Simulate time. A **max-heap** holds the remaining counts of tasks that are ready; a **queue** holds tasks cooling down with the time they become ready again. Each tick: run the task with the most copies left (or idle), and release any task whose cooldown has ended.

Running the most frequent first is the greedy choice — it is the task that most constrains the schedule.

Complexity O(total time · log 26) = O(answer) · O(1) space for 26 letters

C++ SOLUTION

```cpp
class Solution {
public:
    int leastInterval(vector<char>& tasks, int n) {
        int cnt[26] = {0};
        for (char t : tasks) cnt[t - 'A']++;
        priority_queue<int> ready;                    // remaining counts, largest first
        for (int c : cnt) if (c) ready.push(c);
        queue<pair<int, int>> cooling;                // (remaining count, time it is free)
        int time = 0;
        while (!ready.empty() || !cooling.empty()) {
            time++;
            if (!ready.empty()) {                     // run the most-remaining task
                int left = ready.top() - 1; ready.pop();
                if (left) cooling.push({left, time + n});
            }                                         // else: this slot is idle
            if (!cooling.empty() && cooling.front().second == time) {
                ready.push(cooling.front().first);    // cooldown over
                cooling.pop();
            }
        }
        return time;
    }
};
```

WATCH OUT

The O(n) closed form is `max(tasks.size(), (maxCount − 1) · (n + 1) + numberWithMaxCount)` — know it and be ready to explain it with the 'frame' picture. The simulation is what generalises to variants (task order fixed, different cooldowns), so be able to write both.

16 LC 767 · Reorganize String

MEDIUM Pattern 9 — Take the top two

PROBLEM

Rearrange the string so no two adjacent characters are equal, or return "" if impossible.

APPROACH

Max-heap of `(count, char)`. Each round pop the **two** most frequent letters and place both — they are different, so this pair is always safe, and the next round's first letter cannot equal the second letter just placed, because that letter was popped *before* re-pushing.

At the end, at most one letter remains; it is placeable only if its count is 1.

Complexity O(n log σ) time · O(σ) space

C++ SOLUTION

```cpp
class Solution {
public:
    string reorganizeString(string s) {
        int cnt[26] = {0};
        for (char c : s) cnt[c - 'a']++;
        priority_queue<pair<int, char>> pq;
        for (int i = 0; i < 26; i++) if (cnt[i]) pq.push({cnt[i], char('a' + i)});
        string res;
        while (pq.size() >= 2) {                    // place the two most frequent
            auto [f1, c1] = pq.top(); pq.pop();
            auto [f2, c2] = pq.top(); pq.pop();
            res += c1; res += c2;                   // different letters: always safe
            if (--f1) pq.push({f1, c1});
            if (--f2) pq.push({f2, c2});
        }
        if (!pq.empty()) {                          // one letter left over
            if (pq.top().first > 1) return "";      // it would have to repeat
            res += pq.top().second;
        }
        return res;
    }
};
```

WATCH OUT

The quick impossibility check is `maxCount > (n + 1) / 2`. Popping one letter at a time and re-pushing it immediately lets the same letter come out twice in a row — popping two per round is what prevents it.

17 LC 1642 · Furthest Building You Can Reach

MEDIUM Pattern 8 — Regret greedy

PROBLEM

Walk along buildings; each climb needs a ladder or `climb` bricks. Given bricks and ladders, return the furthest index you can reach.

APPROACH

Ladders should cover the **largest** climbs. You cannot know which those are in advance, so put a ladder on every climb tentatively, keeping the ladder climbs in a **min-heap**. When you have used one ladder more than you own, take back the ladder from the **smallest** ladder climb and pay for that climb in bricks. If bricks go negative, you are stuck before the next building.

Complexity O(n log L) for L ladders · O(L) space

C++ SOLUTION

```cpp
class Solution {
public:
    int furthestBuilding(vector<int>& h, int bricks, int ladders) {
        // min-heap of the climbs currently covered by ladders
        priority_queue<int, vector<int>, greater<int>> ladderClimbs;
        for (int i = 0; i + 1 < (int)h.size(); i++) {
            int climb = h[i + 1] - h[i];
            if (climb <= 0) continue;
            ladderClimbs.push(climb);                 // tentatively use a ladder
            if ((int)ladderClimbs.size() > ladders) { // one ladder too many:
                bricks -= ladderClimbs.top();         // pay the SMALLEST climb in bricks
                ladderClimbs.pop();
            }
            if (bricks < 0) return i;                 // cannot reach building i+1
        }
        return (int)h.size() - 1;
    }
};
```

WATCH OUT

Return `i` (the building you are on), not `i + 1`, when bricks run out — you failed to reach `i + 1`. Spending bricks greedily first and ladders later is wrong: it wastes bricks on climbs a ladder should have taken.

18 LC 1834 · Single-Threaded CPU

MEDIUM Pattern 10 — Event simulation

PROBLEM

Tasks arrive at given times with given durations. A single CPU, when free, picks the available task with the shortest duration (ties: smallest index). Return the processing order.

APPROACH

Sort task indices by arrival. Keep a clock and a min-heap of available tasks keyed by `(duration, index)`. Each step: push every task that has arrived by the current time; if none is available, **jump** the clock to the next arrival; otherwise run the top task and advance the clock by its duration.

Complexity O(n log n) time · O(n) space

C++ SOLUTION

```cpp
class Solution {
public:
    vector<int> getOrder(vector<vector<int>>& tasks) {
        int n = tasks.size();
        vector<int> idx(n);
        iota(idx.begin(), idx.end(), 0);
        sort(idx.begin(), idx.end(),
             [&](int a, int b) { return tasks[a][0] < tasks[b][0]; });   // by arrival
        // available tasks: shortest processing time, then smallest index
        priority_queue<pair<int, int>, vector<pair<int, int>>, greater<>> pq;
        vector<int> order;
        long long time = 0;
        int i = 0;
        while ((int)order.size() < n) {
            if (pq.empty() && time < tasks[idx[i]][0])
                time = tasks[idx[i]][0];              // CPU idle: jump ahead
            while (i < n && tasks[idx[i]][0] <= time) {
                pq.push({tasks[idx[i]][1], idx[i]});
                i++;
            }
            auto [proc, id] = pq.top(); pq.pop();
            time += proc;
            order.push_back(id);
        }
        return order;
    }
};
```

WATCH OUT

The clock must be `long long` — 10^5 tasks of duration 10^9 overflow `int`. And jump over idle gaps instead of ticking one unit at a time: arrival times reach 10^9. Sorting the tasks themselves loses the original indices; sort an index array.

19 LC 373 · Find K Pairs with Smallest Sums

MEDIUM Pattern 4 — Implied sorted rows

PROBLEM

Given two sorted arrays, return the k pairs `(a[i], b[j])` with the smallest sums.

APPROACH

Row i is the sorted sequence `a[i]+b[0], a[i]+b[1], …` — so this is a k-way merge over at most k rows. Seed the min-heap with `(a[i]+b[0], i, 0)` for the first k values of i; each pop outputs a pair and pushes the next element of the same row, `(i, j+1)`.

Complexity O(k log k) time · O(k) space

C++ SOLUTION

```cpp
class Solution {
public:
    vector<vector<int>> kSmallestPairs(vector<int>& a, vector<int>& b, int k) {
        // (sum, i, j). Row i is the sorted list a[i]+b[0], a[i]+b[1], ...
        priority_queue<tuple<int, int, int>, vector<tuple<int, int, int>>, greater<>> pq;
        for (int i = 0; i < (int)a.size() && i < k; i++) pq.push({a[i] + b[0], i, 0});
        vector<vector<int>> res;
        while (k-- && !pq.empty()) {
            auto [s, i, j] = pq.top(); pq.pop();
            res.push_back({a[i], b[j]});
            if (j + 1 < (int)b.size()) pq.push({a[i] + b[j + 1], i, j + 1});  // next in row i
        }
        return res;
    }
};
```

WATCH OUT

Generating all n·m pairs is O(nm log nm) and runs out of memory at $10^5 \times 10^5$. Seeding only `(0, 0)` and pushing both `(i+1, j)` and `(i, j+1)` also works but needs a visited set to avoid duplicates — seeding one head per row avoids that entirely.

20 LC 355 · Design Twitter

MEDIUM Pattern 4 — Merge followees' feeds

PROBLEM

Support `postTweet`, `follow`, `unfollow`, and `getNewsFeed` (the 10 most recent tweets from the user and the people they follow).

APPROACH

Store each user's tweets in posting order with a global timestamp. A news feed is a **k-way merge** of the followees' tweet lists, newest first: seed a max-heap with each followee's last tweet `(time, user, index)`, pop up to 10 times, pushing the previous tweet of the same user each time.

> **Complexity** `getNewsFeed`: O(f + 10 log f) for f followees · others O(1)

C++ SOLUTION

```cpp
class Twitter {
    int clock = 0;
    unordered_map<int, vector<pair<int, int>>> tweets;   // user -> (time, tweetId)
    unordered_map<int, unordered_set<int>> follows;
public:
    void postTweet(int userId, int tweetId) {
        tweets[userId].push_back({clock++, tweetId});
    }
    vector<int> getNewsFeed(int userId) {
        // k-way merge of each followee's tweet list, newest first
        priority_queue<tuple<int, int, int>> pq;          // (time, user, index)
        follows[userId].insert(userId);                   // see your own tweets
        for (int u : follows[userId]) {
            auto& t = tweets[u];
            if (!t.empty()) pq.push({t.back().first, u, (int)t.size() - 1});
        }
        vector<int> feed;
        while (!pq.empty() && feed.size() < 10) {
            auto [time, u, i] = pq.top(); pq.pop();
            feed.push_back(tweets[u][i].second);
            if (i > 0) pq.push({tweets[u][i - 1].first, u, i - 1});
        }
        return feed;
    }
    void follow(int a, int b)   { follows[a].insert(b); }
    void unfollow(int a, int b) { if (a != b) follows[a].erase(b); }
};
```

WATCH OUT

A user always sees their own tweets — insert `userId` into their own follow set, and make `unfollow` refuse to remove it. Copying and sorting every followee's full tweet list per feed request is O(T log T) and is what the interviewer is steering you away from.

21 LC 264 · Ugly Number II

MEDIUM Pattern 4 — Generate successors

PROBLEM

Return the n-th positive integer whose only prime factors are 2, 3 and 5.

APPROACH

Every ugly number is 2, 3 or 5 times a smaller ugly number. Start a min-heap with 1; pop the smallest, and push its three multiples. The n-th distinct pop is the answer.

The same number is generated several ways (6 = 2·3 = 3·2), so drop equal values when popping.

Complexity O(n log n) time · O(n) space

C++ SOLUTION

```cpp
class Solution {
public:
    int nthUglyNumber(int n) {
        priority_queue<long long, vector<long long>, greater<long long>> pq;
        pq.push(1);
        long long cur = 1;
        for (int i = 0; i < n; i++) {
            cur = pq.top(); pq.pop();
            while (!pq.empty() && pq.top() == cur) pq.pop();   // drop duplicates
            for (int f : {2, 3, 5}) pq.push(cur * f);           // generate successors
        }
        return (int)cur;
    }
};
```

WATCH OUT

Use `long long` in the heap — the pushed multiples of the 1690th ugly number exceed `int`. The O(n) answer is three pointers into the output array (the DP version): know it as the follow-up. Forgetting to skip duplicates returns a repeated number as a later answer.

22 LC 658 · Find K Closest Elements

MEDIUM Pattern 2 — Distance with a tie-break

PROBLEM

Given a sorted array, return the k elements closest to x, in ascending order. On a distance tie, the smaller element is closer.

APPROACH

Max-heap of `(|v − x|, v)` of size k. The pair ordering makes the farthest — and, on a tie, the **larger** value — the top, so it is evicted first, which is exactly the tie rule. Sort the k survivors at the end.

Complexity O(n log k) time · O(k) space

C++ SOLUTION

```cpp
class Solution {
public:
    vector<int> findClosestElements(vector<int>& arr, int k, int x) {
        // max-heap on (distance, value): the top is the WORST of the k kept,
        // and a larger value loses a distance tie, as the problem requires
        priority_queue<pair<int, int>> pq;
        for (int v : arr) {
            pq.push({abs(v - x), v});
            if ((int)pq.size() > k) pq.pop();
        }
        vector<int> res;
        while (!pq.empty()) { res.push_back(pq.top().second); pq.pop(); }
        sort(res.begin(), res.end());
        return res;
    }
};
```

WATCH OUT

The input is **sorted**, so the optimal solution is a binary search for the window's left edge — O(log(n − k) + k). The heap version is the general one for unsorted input; say both. Check the tie rule on `[1,3]`, k = 1, x = 2: the answer is `[1]`.

23 LC 846 · Hand of Straights

MEDIUM Pattern 1 — The smallest must start a run

PROBLEM

Can the cards be split into groups of `groupSize` consecutive values?

APPROACH

The smallest card can only be the **start** of a group — nothing smaller exists to precede it. So repeatedly take the smallest remaining value from a min-heap of distinct values and consume one of each of the next `groupSize` values from a count map.

When a value's count drops to zero it must be the current heap minimum; if it is not, a smaller value is stranded and the answer is false.

Complexity O(n log n) time · O(n) space

C++ SOLUTION

```cpp
class Solution {
public:
    bool isNStraightHand(vector<int>& hand, int groupSize) {
        if (hand.size() % groupSize) return false;
        unordered_map<int, int> cnt;
        for (int c : hand) cnt[c]++;
        priority_queue<int, vector<int>, greater<int>> pq;   // distinct values
        for (auto& [v, c] : cnt) pq.push(v);
        while (!pq.empty()) {
            int first = pq.top();                    // smallest card MUST start a group
            for (int v = first; v < first + groupSize; v++) {
                if (cnt[v] == 0) return false;       // a gap in the run
                if (--cnt[v] == 0) {
                    // a value can only run out if it is the current minimum;
                    // otherwise a smaller value is left with no way to finish
                    if (v != pq.top()) return false;
                    pq.pop();
                }
            }
        }
        return true;
    }
};
```

WATCH OUT

Check `hand.size() % groupSize` first. The stranded-value check is the subtle line: without it, a value can hit zero while a smaller one still has copies, and that smaller one can never finish a run. A `std::map` instead of heap + hash map gives the same algorithm with sorted iteration.

24 LC 743 · Network Delay Time

MEDIUM Pattern 11 — Dijkstra

PROBLEM

A signal starts at node k of a weighted directed graph. Return the time for all nodes to receive it, or −1 if some never do.

APPROACH

Single-source shortest paths with non-negative weights — Dijkstra. A min-heap of `(dist, node)`; pop the closest unfinished node, relax its edges, push improved distances as **new** entries. The answer is the largest finite distance.

Complexity O((V + E) log V) time · O(V + E) space

C++ SOLUTION

```cpp
class Solution {
public:
    int networkDelayTime(vector<vector<int>>& times, int n, int k) {
        vector<vector<pair<int, int>>> adj(n + 1);           // u -> (v, w)
        for (auto& t : times) adj[t[0]].push_back({t[1], t[2]});
        vector<int> dist(n + 1, INT_MAX);
        priority_queue<pair<int, int>, vector<pair<int, int>>, greater<>> pq;  // (dist, node)
        dist[k] = 0;
        pq.push({0, k});
        while (!pq.empty()) {
            auto [d, u] = pq.top(); pq.pop();
            if (d > dist[u]) continue;                       // stale entry: skip
            for (auto [v, w] : adj[u])
                if (d + w < dist[v]) {
                    dist[v] = d + w;
                    pq.push({dist[v], v});
                }
        }
        int best = *max_element(dist.begin() + 1, dist.end());
        return best == INT_MAX ? -1 : best;
    }
};
```

WATCH OUT

`if (d > dist[u]) continue;` skips stale entries — the lazy-deletion substitute for decrease-key. Without it the result is still correct but slower. And return −1 if any `dist` is still infinite — the graph may be disconnected.

25 LC 1353 · Maximum Number of Events That Can Be Attended

MEDIUM Pattern 6 — Earliest deadline among the available

PROBLEM

Each event spans days `[start, end]`; you may attend one event per day. Maximise the number of events attended.

APPROACH

Sweep day by day. Events that have started join a **min-heap of end days**. Each day attend the available event that **ends soonest** — the one most at risk of being missed — then discard events that have already ended. When nothing is available, jump to the next start day.

Complexity O(n log n) time — each loop iteration attends one event · O(n) space

C++ SOLUTION

```cpp
class Solution {
public:
    int maxEvents(vector<vector<int>>& events) {
        sort(events.begin(), events.end());                 // by start day
        priority_queue<int, vector<int>, greater<int>> ends; // open events' end days
        int i = 0, n = events.size(), day = 0, attended = 0;
        while (i < n || !ends.empty()) {
            if (ends.empty()) day = max(day, events[i][0]);  // skip idle days
            while (i < n && events[i][0] <= day) ends.push(events[i++][1]);
            ends.pop();                                      // attend the one ending first
            attended++;
            day++;
            while (!ends.empty() && ends.top() < day) ends.pop();   // expired
        }
        return attended;
    }
};
```

WATCH OUT

Attending the event that *started* first, or the shortest one, is wrong; earliest **end** is the exchange-argument choice. Remember to throw away expired events before each attendance, or you will 'attend' an event on a day after it ended.

26 LC 1405 · Longest Happy String

MEDIUM Pattern 9 — Hold one back

PROBLEM

Build the longest string using at most a 'a's, b 'b's and c 'c's, with no three equal letters in a row.

APPROACH

Max-heap of `(remaining, letter)`. Normally append the most plentiful letter. If that would make three in a row, append **one** of the runner-up instead and put the leader back untouched. If there is no runner-up, stop.

Complexity O(a + b + c) time (heap of size ≤ 3) · O(1) extra space

C++ SOLUTION

```cpp
class Solution {
public:
    string longestDiverseString(int a, int b, int c) {
        priority_queue<pair<int, char>> pq;
        if (a) pq.push({a, 'a'});
        if (b) pq.push({b, 'b'});
        if (c) pq.push({c, 'c'});
        string s;
        while (!pq.empty()) {
            auto [f1, c1] = pq.top(); pq.pop();
            int n = s.size();
            if (n >= 2 && s[n - 1] == c1 && s[n - 2] == c1) {   // would make "ccc"
                if (pq.empty()) break;                          // nothing else to use
                auto [f2, c2] = pq.top(); pq.pop();             // use the runner-up once
                s += c2;
                if (--f2) pq.push({f2, c2});
                pq.push({f1, c1});                              // the leader waits
            } else {
                s += c1;
                if (--f1) pq.push({f1, c1});
            }
        }
        return s;
    }
};
```

WATCH OUT

When the leader is blocked, use the runner-up **once**, not as many as possible — the leader should resume as soon as it legally can, since it is the one that limits the length. Check the last two characters, not just the last one.

27 LC 871 · Minimum Number of Refueling Stops

HARD Pattern 8 — Retroactive greedy

PROBLEM

Drive to `target` with `startFuel` and stations at given positions with given fuel. Return the fewest refuelling stops, or −1.

APPROACH

Drive as far as the current fuel allows, pushing the fuel of every station passed into a max-heap. When you cannot reach further, pretend you had stopped at the **best** station passed: pop the largest and add it to the reach. Count the pops.

Complexity O(n log n) time · O(n) space

C++ SOLUTION

```cpp
class Solution {
public:
    int minRefuelStops(int target, int fuel, vector<vector<int>>& stations) {
        priority_queue<int> passed;          // fuel of stations we drove past
        long long reach = fuel;
        int stops = 0, i = 0, n = stations.size();
        while (reach < target) {
            while (i < n && stations[i][0] <= reach) passed.push(stations[i++][1]);
            if (passed.empty()) return -1;   // stranded
            reach += passed.top();           // "retroactively" stop at the best one
            passed.pop();
            stops++;
        }
        return stops;
    }
};
```

WATCH OUT

It feels like cheating, but it is valid: the decision *which* stations to stop at only matters when fuel runs out, and at that moment the best one passed is always the right choice. Return −1 when the heap is empty and the target is still out of reach.

28 LC 1882 · Process Tasks Using Servers

MEDIUM Pattern 10 — Free and busy heaps

PROBLEM

Task j arrives at second j. It goes to the free server with the smallest weight (then smallest index). If none is free, it waits for the earliest-freed one. Return the server for each task.

APPROACH

Two heaps. `free`: `(weight, index)`. `busy`: `(freeAt, weight, index)`. For each task, set the clock to `max(clock, j)`; if no server is free, jump the clock to the earliest `freeAt`. Move every server freed by now from `busy` to `free`, then assign the top of `free`.

Complexity O((n + m) log n) time · O(n) space

C++ SOLUTION

```cpp
class Solution {
public:
    vector<int> assignTasks(vector<int>& servers, vector<int>& tasks) {
        using T = tuple<long long, int, int>;
        priority_queue<pair<int, int>, vector<pair<int, int>>, greater<>> freeQ;  // (weight, idx)
        priority_queue<T, vector<T>, greater<>> busy;              // (freeAt, weight, idx)
        for (int i = 0; i < (int)servers.size(); i++) freeQ.push({servers[i], i});
        vector<int> res(tasks.size());
        long long time = 0;
        for (int j = 0; j < (int)tasks.size(); j++) {
            time = max(time, (long long)j);          // task j arrives at second j
            if (freeQ.empty()) time = get<0>(busy.top());   // wait for the first release
            while (!busy.empty() && get<0>(busy.top()) <= time) {
                auto [t, w, i] = busy.top(); busy.pop();
                freeQ.push({w, i});
            }
            auto [w, i] = freeQ.top(); freeQ.pop();
            res[j] = i;
            busy.push({time + tasks[j], w, i});
        }
        return res;
    }
};
```

WATCH OUT

When several servers free up at the same moment, the choice among them is by weight and index — which is why they must all be moved into `free` before choosing, not taken straight from `busy`. The clock is `long long`: waiting times accumulate.

Tier 3 — Harder, but they do appear

Two halves with deletion, locked-versus-available pools, regret greedy with a sort, minimax Dijkstra and lazy deletion. Attempt these once Tier 2 is comfortable.

29 LC 480 · Sliding Window Median

HARD Pattern 12 — Two halves with deletion

PROBLEM

Return the median of every window of size k.

APPROACH

LC 295's two halves, but elements must also **leave**, and `priority_queue` cannot erase. So keep each half as a `multiset` — the heap you can erase from — and rebalance exactly as in LC 295.

Complexity O(n log k) time · O(k) space

C++ SOLUTION

```cpp
class Solution {
    // LC 295's two halves, but a window needs DELETION -> multisets,
    // the "heap you can erase from".  lo.size() == hi.size() or one more.
    multiset<int> lo, hi;
    void balance() {          // one move is enough: sizes drift by at most 2
        if (lo.size() > hi.size() + 1) { hi.insert(*lo.rbegin()); lo.erase(prev(lo.end())); }
        else if (lo.size() < hi.size()) { lo.insert(*hi.begin()); hi.erase(hi.begin()); }
    }
public:
    vector<double> medianSlidingWindow(vector<int>& nums, int k) {
        vector<double> res;
        for (int i = 0; i < (int)nums.size(); i++) {
            int x = nums[i];
            if (lo.empty() || x <= *lo.rbegin()) lo.insert(x); else hi.insert(x);
            if (i >= k) {                          // slide: erase ONE copy of the oldest
                int y = nums[i - k];
                auto it = lo.find(y);
                if (it != lo.end()) lo.erase(it); else hi.erase(hi.find(y));
            }
            balance();
            if (i >= k - 1)
                res.push_back(k % 2 ? *lo.rbegin() : ((double)*lo.rbegin() + *hi.begin()) / 2.0);
        }
        return res;
    }
};
```

WATCH OUT

Erase **one** copy with `erase(find(x))` — `erase(x)` removes every duplicate. Average in `double`: `INT_MAX + INT_MAX` overflows.

30 LC 502 · IPO

HARD Pattern 6 — Locked vs available

PROBLEM

Starting with capital w, finish at most k projects; each needs a minimum capital and adds its profit. Maximise the final capital.

APPROACH

Sort projects by required capital. Keep a pointer: every project whose requirement is now met moves into a **max-heap of profits**. Then take the most profitable available project. Capital only grows, so a project never becomes locked again.

Complexity O(n log n) time · O(n) space

C++ SOLUTION

```cpp
class Solution {
public:
    int findMaximizedCapital(int k, int w, vector<int>& profits, vector<int>& capital) {
        int n = profits.size();
        vector<pair<int, int>> proj(n);                    // (capital needed, profit)
        for (int i = 0; i < n; i++) proj[i] = {capital[i], profits[i]};
        sort(proj.begin(), proj.end());
        priority_queue<int> affordable;                    // profits we can start now
        int i = 0;
        while (k--) {
            while (i < n && proj[i].first <= w) affordable.push(proj[i++].second);
            if (affordable.empty()) break;                 // nothing affordable, ever
            w += affordable.top();                         // greedy: most profitable
            affordable.pop();
        }
        return w;
    }
};
```

WATCH OUT

Stop early when the heap is empty — no remaining project is affordable now, and since capital cannot grow without a project, none ever will be. Re-scanning all projects every round is O(nk).

31 LC 630 · Course Schedule III

HARD Pattern 8 — Drop the longest

PROBLEM

Each course has a duration and a last day. Take courses one at a time from day 1. Maximise the number taken.

APPROACH

Process courses by **deadline**. Take each one, adding its duration to the running time and to a max-heap. If the running time now exceeds this course's deadline, drop the **longest** course taken so far (possibly this one). The count never decreases, and the total time shrinks as much as possible.

Complexity O(n log n) time · O(n) space

C++ SOLUTION

```cpp
class Solution {
public:
    int scheduleCourse(vector<vector<int>>& courses) {
        sort(courses.begin(), courses.end(),
             [](auto& a, auto& b) { return a[1] < b[1]; });   // earliest deadline first
        priority_queue<int> taken;                             // durations, longest on top
        int time = 0;
        for (auto& c : courses) {
            taken.push(c[0]);
            time += c[0];
            if (time > c[1]) {                // deadline broken: drop the longest course
                time -= taken.top();          // (it may be the one just added)
                taken.pop();
            }
        }
        return taken.size();
    }
};
```

WATCH OUT

Sorting by duration instead of deadline gives wrong answers. And when the deadline breaks, drop the **longest** course, not the most recent — swapping a long course for a short one keeps the count and frees the most time for later courses.

32 LC 857 · Minimum Cost to Hire K Workers

HARD Pattern 8 — Sort by ratio, evict the heaviest

PROBLEM

Hire exactly k workers. Pay is proportional to quality within the group, and each worker must get at least their minimum wage. Minimise the total pay.

APPROACH

In any group, the worker with the highest `wage/quality` ratio sets the rate, and the cost is `rate × sum of qualities`. Sort workers by ratio. Walking in that order, the current worker is the rate-setter; keep the k **smallest qualities** seen so far with a max-heap of qualities and a running sum.

Complexity O(n log n) time · O(n) space

C++ SOLUTION

```cpp
class Solution {
public:
    double mincostToHireWorkers(vector<int>& quality, vector<int>& wage, int k) {
        int n = quality.size();
        vector<pair<double, int>> w(n);                    // (wage/quality, quality)
        for (int i = 0; i < n; i++) w[i] = {(double)wage[i] / quality[i], quality[i]};
        sort(w.begin(), w.end());                          // by ratio, ascending
        priority_queue<int> pq;                            // qualities, largest on top
        long long qsum = 0;
        double best = 1e18;
        for (auto& [ratio, q] : w) {
            pq.push(q); qsum += q;
            if ((int)pq.size() > k) { qsum -= pq.top(); pq.pop(); }   // drop the heaviest
            // this worker has the highest ratio so far, so it sets everyone's pay rate
            if ((int)pq.size() == k) best = min(best, ratio * qsum);
        }
        return best;
    }
};
```

WATCH OUT

Evaluate the cost only once the heap holds exactly k workers. The worker just added may be the one evicted — that is fine, because that candidate cost is then computed with a rate at least as high as needed and can only be an overestimate, never a false minimum.

33 LC 1383 · Maximum Performance of a Team

HARD Pattern 8 — Sort by efficiency, evict the slowest

PROBLEM

Choose at most k engineers to maximise `(sum of speeds) × (minimum efficiency)`. Return it modulo $10^9 + 7$.

APPROACH

Sort by efficiency **descending**. Walking in that order, the current engineer has the minimum efficiency of any team built from engineers seen so far. Keep the fastest k speeds with a min-heap and a running sum; evaluate `sum × efficiency` at every step.

Complexity O(n log n) time · O(n) space

C++ SOLUTION

```cpp
class Solution {
public:
    int maxPerformance(int n, vector<int>& speed, vector<int>& efficiency, int k) {
        vector<pair<int, int>> eng(n);                          // (efficiency, speed)
        for (int i = 0; i < n; i++) eng[i] = {efficiency[i], speed[i]};
        sort(eng.rbegin(), eng.rend());                         // efficiency, descending
        priority_queue<int, vector<int>, greater<int>> speeds; // min-heap of chosen speeds
        long long sum = 0, best = 0;
        for (auto& [e, s] : eng) {
            speeds.push(s); sum += s;
            if ((int)speeds.size() > k) { sum -= speeds.top(); speeds.pop(); }
            best = max(best, sum * e);          // e is the minimum efficiency of the team
        }
        return best % 1000000007;               // modulo only AFTER maximising
    }
};
```

WATCH OUT

Maximise in `long long` and apply the modulo **only to the final answer** — reducing inside the loop compares wrong numbers. 'At most k' means you evaluate at every size up to k, which this loop does automatically.

34 LC 632 · Smallest Range Covering Elements from K Lists

HARD Pattern 4 — K-way merge + running max

PROBLEM

Find the smallest range `[a, b]` containing at least one number from each of k sorted lists.

APPROACH

Keep one pointer per list. The heap gives the current minimum across the pointers; track the current maximum separately. `[min, max]` covers every list — record it. The only way to possibly shrink it is to raise the minimum, so advance that list's pointer. Stop when any list is exhausted.

Complexity O(N log k) for N total numbers · O(k) space

C++ SOLUTION

```cpp
class Solution {
public:
    vector<int> smallestRange(vector<vector<int>>& nums) {
        // one pointer per list; the heap gives the current minimum, and we
        // track the current maximum by hand
        priority_queue<tuple<int, int, int>, vector<tuple<int, int, int>>, greater<>> pq;
        int curMax = INT_MIN;
        for (int i = 0; i < (int)nums.size(); i++) {
            pq.push({nums[i][0], i, 0});
            curMax = max(curMax, nums[i][0]);
        }
        vector<int> best{get<0>(pq.top()), curMax};
        while (true) {
            auto [v, i, j] = pq.top(); pq.pop();
            if (curMax - v < best[1] - best[0]) best = {v, curMax};
            if (j + 1 == (int)nums[i].size()) break;      // a list ran out: done
            int nv = nums[i][j + 1];
            pq.push({nv, i, j + 1});
            curMax = max(curMax, nv);
        }
        return best;
    }
};
```

WATCH OUT

A heap gives only the min, so the max must be maintained by hand — and it only ever grows. Stop as soon as the popped list has no next element: from then on no range can include that list. Ties are broken by the smaller left endpoint, which the strict `<` comparison preserves.

35 LC 407 · Trapping Rain Water II

HARD Pattern 11 — The wall shrinks from its lowest point

PROBLEM

Given a 2-D height map, compute how much water it can trap.

APPROACH

Water spills over the **lowest** point of the surrounding wall. Start with the border as the wall in a min-heap. Pop the lowest wall cell; each unvisited neighbour holds `max(0, level − height)` water and joins the wall at height `max(level, height)`. The wall moves inward, always from its lowest point.

Complexity O(R · C · log(R · C)) time · O(R · C) space

C++ SOLUTION

```cpp
class Solution {
public:
    int trapRainWater(vector<vector<int>>& h) {
        int R = h.size(), C = h[0].size();
        priority_queue<tuple<int, int, int>, vector<tuple<int, int, int>>, greater<>> pq;
        vector<vector<bool>> seen(R, vector<bool>(C, false));
        for (int r = 0; r < R; r++)
            for (int c = 0; c < C; c++)
                if (r == 0 || c == 0 || r == R - 1 || c == C - 1) {   // the outer wall
                    pq.push({h[r][c], r, c});
                    seen[r][c] = true;
                }
        int water = 0;
        int dr[] = {1, -1, 0, 0}, dc[] = {0, 0, 1, -1};
        while (!pq.empty()) {
            auto [level, r, c] = pq.top(); pq.pop();   // lowest point of the wall
            for (int d = 0; d < 4; d++) {
                int nr = r + dr[d], nc = c + dc[d];
                if (nr < 0 || nc < 0 || nr >= R || nc >= C || seen[nr][nc]) continue;
                seen[nr][nc] = true;
                water += max(0, level - h[nr][nc]);    // fills up to the wall's low point
                pq.push({max(level, h[nr][nc]), nr, nc});   // wall moves inward
            }
        }
        return water;
    }
};
```

WATCH OUT

The two-pointer solution of LC 42 does not generalise to 2-D — say that, then explain why the heap does. Push the neighbour with the **raised** height `max(level, h)`, not its own height, or water behind it is miscounted.

36 LC 778 · Swim in Rising Water

HARD Pattern 11 — Minimax Dijkstra

PROBLEM

At time t you can swim through cells of elevation ≤ t. Return the earliest time you can reach the bottom-right corner from the top-left.

APPROACH

The cost of a path is its **highest** cell. Dijkstra works with `max` in place of `+`: a min-heap of `(costSoFar, r, c)`, and moving to a neighbour costs `max(costSoFar, grid[nr][nc])`. The first time the target is popped, its cost is optimal.

Complexity $O(n^2 \log n)$ time · $O(n^2)$ space

C++ SOLUTION

```cpp
class Solution {
public:
    int swimInWater(vector<vector<int>>& g) {
        int n = g.size();
        // Dijkstra where a path's cost is its MAXIMUM cell, not its sum
        priority_queue<tuple<int, int, int>, vector<tuple<int, int, int>>, greater<>> pq;
        vector<vector<bool>> seen(n, vector<bool>(n, false));
        pq.push({g[0][0], 0, 0});
        seen[0][0] = true;
        int dr[] = {1, -1, 0, 0}, dc[] = {0, 0, 1, -1};
        while (true) {
            auto [t, r, c] = pq.top(); pq.pop();
            if (r == n - 1 && c == n - 1) return t;
            for (int d = 0; d < 4; d++) {
                int nr = r + dr[d], nc = c + dc[d];
                if (nr < 0 || nc < 0 || nr >= n || nc >= n || seen[nr][nc]) continue;
                seen[nr][nc] = true;
                pq.push({max(t, g[nr][nc]), nr, nc});
            }
        }
    }
};
```

WATCH OUT

Include `grid[0][0]` in the starting cost — you cannot start before the start cell is under water. Alternatives worth naming: binary search on t with a BFS, and union-find adding cells in increasing elevation.

37 LC 1675 · Minimize Deviation in Array

HARD Pattern 1 — Normalise, then shrink the max

PROBLEM

You may double odd numbers and halve even numbers, any number of times. Minimise the difference between the maximum and the minimum.

APPROACH

Odd numbers can only go up once; even numbers can only go down. So first push every number to its **maximum** (double the odds). Now the only useful move is halving the current maximum. Use a max-heap, track the minimum, and record `max − min` each step. Stop when the maximum is odd — it can no longer shrink.

Complexity O(n log n · log M) for maximum value M · O(n) space

C++ SOLUTION

```cpp
class Solution {
public:
    int minimumDeviation(vector<int>& nums) {
        // make every number as LARGE as it can be (odd -> doubled once);
        // from then on the only legal move is halving an even number
        priority_queue<int> pq;
        int mn = INT_MAX;
        for (int x : nums) {
            if (x % 2) x *= 2;
            pq.push(x);
            mn = min(mn, x);
        }
        int best = INT_MAX;
        while (true) {
            int mx = pq.top(); pq.pop();
            best = min(best, mx - mn);
            if (mx % 2) break;              // the max is odd: it can never shrink
            mx /= 2;                         // shrink the max, the only useful move
            mn = min(mn, mx);
            pq.push(mx);
        }
        return best;
    }
};
```

WATCH OUT

Halving anything other than the maximum cannot reduce the deviation. Update the running minimum when you push the halved value — it may become the new minimum. The odd-maximum stop is what terminates the loop.

38 LC 218 · The Skyline Problem

HARD Pattern 12 — Lazy deletion

PROBLEM

Given buildings `[left, right, height]` sorted by left edge, return the key points of the skyline.

APPROACH

Process events in x order: the next event is either the next building's left edge or the right edge of the tallest live building. Live buildings sit in a max-heap of `(height, right)`. At a right edge, pop every building that has ended — buildings that ended while hidden underneath are removed lazily when they surface. After each event, the skyline height is the heap's top (or 0); output a point when it changes.

Complexity O(n log n) time · O(n) space

C++ SOLUTION

```cpp
class Solution {
public:
    vector<vector<int>> getSkyline(vector<vector<int>>& b) {   // b sorted by left edge
        vector<vector<int>> res;
        priority_queue<pair<int, int>> live;                    // (height, right edge)
        int i = 0, n = b.size();
        while (i < n || !live.empty()) {
            int x;
            if (live.empty() || (i < n && b[i][0] <= live.top().second)) {
                x = b[i][0];                                    // next event: a left edge
                while (i < n && b[i][0] == x) { live.push({b[i][2], b[i][1]}); i++; }
            } else {
                x = live.top().second;                          // next event: tallest ends
                while (!live.empty() && live.top().second <= x) live.pop();  // lazy delete
            }
            int h = live.empty() ? 0 : live.top().first;
            if (res.empty() || res.back()[1] != h) res.push_back({x, h});
        }
        return res;
    }
};
```

WATCH OUT

Push **all** buildings that start at the same x before reading the height, and drop a key point when the height does not change — otherwise you emit duplicate or zero-width points. The alternative is a sorted event list with a `multiset` of heights, erasing exactly at each right edge.